University of Moratuwa

Department of Electronic and Telecommunication Engineering



Resource-Constrained CNN for Edge Image Classification

EN3150 Pattern Recognition

Group Name:	**Rubix**
230264R	Janurajah J.
230585C	Saruka U.
230638T	Thayarathan V.

Submitted on	October 19, 2026

Contents

1 Introduction **1**

2 Dataset and Preprocessing **1**
2.1 Dataset description 1
2.2 Data split 1
2.3 Preprocessing pipeline 2

3 Custom CNN Architecture Design **3**
3.1 Model A: Standard CNN 3
3.2 Model B: Lightweight CNN 3
3.3 Activation functions and hardware considerations 4

4 Optimizer Selection and Training Methodology **5**
4.1 Optimizer comparison 5
4.2 Training configuration 6

5 Experimental Results **7**
5.1 Model A learning curves 7
5.2 Model B learning curves 7
5.3 Learning Curve Analysis 7
5.4 Test-set classification performance 7
5.5 Confusion matrices 8
5.6 Pretrained model learning curves 10

6 Resource Efficiency and Model Comparison **13**
6.1 Custom model comparison 13
6.2 Comparison with lightweight pretrained networks 13
6.3 Accuracy, memory and computation trade-offs 13

7 Limitations **15**

8 Conclusion **15**

9 Reproducibility and GitHub Repository **15**

10 References **16**

1 Introduction

Convolutional neural networks (CNNs) are widely used for image classification, but the memory and computational requirements of large models can make deployment difficult on embedded devices and edge nodes. This assignment investigates the trade-off between classification performance and resource efficiency by developing two custom CNNs and comparing them with two pretrained lightweight architectures.

Fashion-MNIST is used as the image-classification dataset. Model A uses conventional two-dimensional convolutions, while Model B uses depthwise separable convolutions and is constrained to no more than 100,000 trainable parameters. MobileNetV2 and EfficientNetB0 are fine-tuned as pretrained reference models.

The objectives are to prepare low-resolution data with a 70/15/15 split; design and document two custom CNN architectures; compare Adam, SGD and SGD with Momentum for Model B; evaluate all models using classification metrics and resource measures; and discuss accuracy, memory footprint and computational cost.

2 Dataset and Preprocessing

2.1 Dataset description

Fashion-MNIST contains 70,000 grayscale images of clothing items across ten classes. Each original image is 28×28 pixels. Table 1 lists the labels.

Table 1: Fashion-MNIST class labels

Label	Class name
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

2.2 Data split

The data were split into training, validation and test subsets using a 70/15/15 ratio. The processed dataset used in all five notebooks contains 70,000 examples split into 49,000 training images, 10,500 validation images and 10,500 test images. The proportions are therefore 70%, 15% and 15%.

Table 2: Dataset split used in the experiments

Subset	Percentage	Number of images
Training	70%	49,000
Validation	15%	10,500
Testing	15%	10,500
Total	100%	70,000

2.3 Preprocessing pipeline

For the custom CNNs, images are represented as single-channel grayscale inputs at 28 $\times$ 28 pixels, normalized to the range $[0, 1]$. MobileNetV2 and EfficientNetB0 use images resized to 64 $\times$ 64 pixels and converted to three channels. The same underlying training, validation and test examples are used for all models.

For MobileNetV2, the notebook applies the Keras MobileNetV2 preprocessing function. The EfficientNetB0 Keras application includes its own rescaling layer, so the notebook passes the resized float32 image values without applying the MobileNetV2 transformation. The pipelines are therefore model-specific.

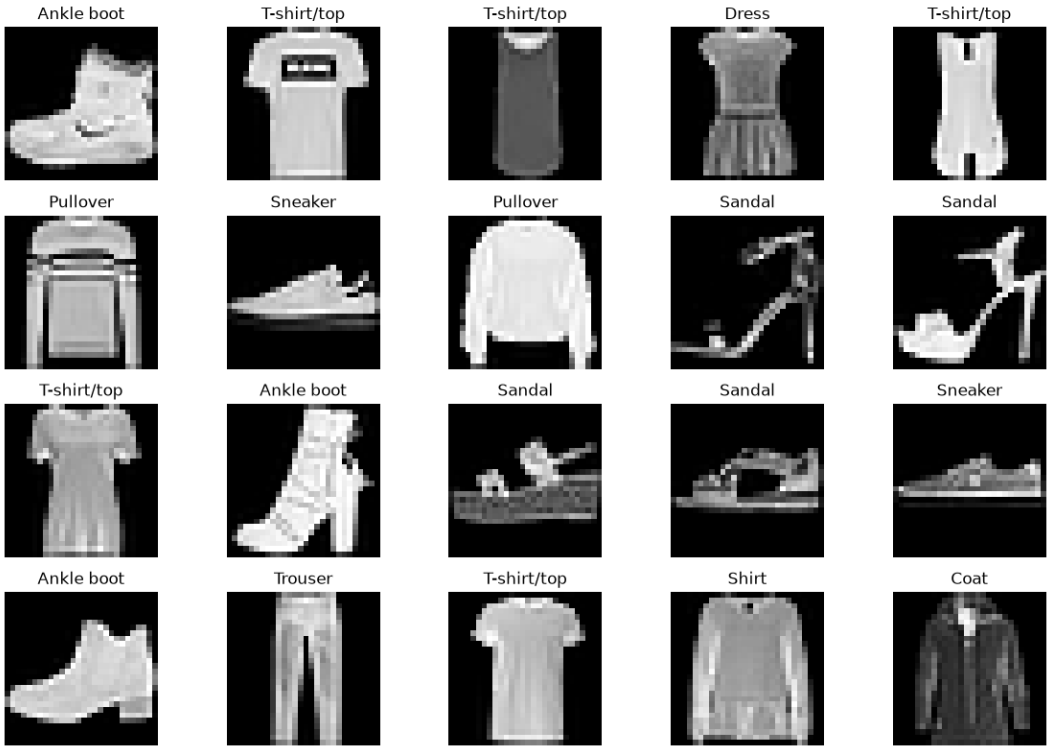


Figure 1: Representative Fashion-MNIST images with class labels.

3 Custom CNN Architecture Design

3.1 Model A: Standard CNN

Model A is the baseline CNN. Conventional convolutional layers extract spatial features, and max-pooling layers reduce feature-map dimensions. The resulting representation is flattened and passed to a dense hidden layer and a ten-class softmax output.

Table 3: Model A architecture and parameter counts from the executed model summary.

Layer	Configuration	Parameters
Input	$28 \times 28 \times 1$	0
Conv2D	16 filters, 3×3, ReLU	160
MaxPooling2D	2×2	0
Conv2D	32 filters, 3×3, ReLU	4,640
MaxPooling2D	2×2	0
Conv2D	64 filters, 3×3, ReLU	18,496
MaxPooling2D	2×2	0
Flatten	Flatten feature maps	0
Dense	64 units, ReLU	36,928
Dense	10 units, softmax	650
Total		60,874

For a conventional convolution with biases, the parameter count is

$$P_{\text{conv}} = (K_h K_w C_{\text{in}} + 1) C_{\text{out}}, \tag{1}$$

where K_h and K_w are kernel dimensions, C_{in} is the number of input channels and C_{out} is the number of filters. For a dense layer,

$$P_{\text{dense}} = (n_{\text{in}} + 1) n_{\text{out}}. \tag{2}$$

3.2 Model B: Lightweight CNN

Model B replaces conventional convolutions with depthwise separable convolutions. A depthwise convolution applies a spatial filter independently to each input channel; a pointwise 1×1 convolution combines the channel-wise features. This factorization can reduce parameter count and multiply-accumulate operations relative to a conventional convolution with comparable channel dimensions.

For a depthwise separable convolution with a $K_h \times K_w$ kernel, C_{in} input channels and C_{out} output channels, with one bias per output channel,

$$P_{\text{sep}} = K_h K_w C_{\text{in}} + C_{\text{in}} C_{\text{out}} + C_{\text{out}}. \tag{3}$$

Table 4: Model B lightweight CNN architecture, grouped by functional block

Layer/block	Configuration	Parameters
Input	$28 \times 28 \times 1$	0
Stem convolution	3×3, 16 channels, no bias; batch normalization and ReLU	208
Max pooling	2×2	0
Depthwise-separable block 1	Depthwise 3×3 (16 channels), pointwise 1×1 to 24 channels; batch normalization and ReLU	688
Max pooling	2×2	0
Depthwise-separable block 2	Depthwise 3×3 (24 channels), pointwise 1×1 to 32 channels; batch normalization and ReLU	1,208
Max pooling	2×2	0
Depthwise-separable block 3	Depthwise 3×3 (32 channels), pointwise 1×1 to 48 channels; batch normalization and ReLU	2,144
Global average pooling and dropout	Dropout rate 0.20	0
Dense output	10 units, softmax	490
Total	4,354 trainable and 384 non-trainable parameters	4,738

3.3 Activation functions and hardware considerations

ReLU is used in hidden layers because $f(x) = \max(0, x)$ is computationally simple and avoids the exponential calculations used by sigmoid and tanh. It is widely supported by deep-learning runtimes. The output layer uses softmax to convert class logits into a probability distribution over ten classes. Softmax requires exponentials but is appropriate for normalized multi-class probabilities; its cost should be considered separately from the main convolutional computation.

4 Optimizer Selection and Training Methodology

4.1 Optimizer comparison

Model B's optimizer selection compares Adam, standard stochastic gradient descent (SGD), and SGD with Momentum. Candidates should use the same data splits, architecture, batch size and comparable training settings. Select the optimizer using validation metrics, not test-set performance.

Table 5: Optimizer comparison using the recorded validation results.

Optimizer	Learning rate	Final validation accuracy (%)	Final validation loss	Time (s/epoch)
Adam	0.001	87.09	0.3533	6.90
SGD	0.01	84.88	0.4237	6.87
SGD with Momentum	0.01	86.50	0.3700	7.31

SGD updates parameters using the current gradient. Momentum incorporates a contribution from previous gradients, which can smooth updates and accelerate progress in consistent directions. Adam adapts update scales using estimates of first and second gradient moments. The methods may differ in convergence speed, stability and validation performance.

Selected optimizer and learning rate: Adam, learning rate 10^{-3}.
Reason for selection: Adam achieved the highest final validation accuracy (87.09%) and the lowest final validation loss (0.3533) in the recorded comparison. SGD with Momentum improved validation accuracy over plain SGD (86.50% versus 84.88%), but that run completed only 11 epochs, so its endpoint is not directly comparable with the 20-epoch runs.

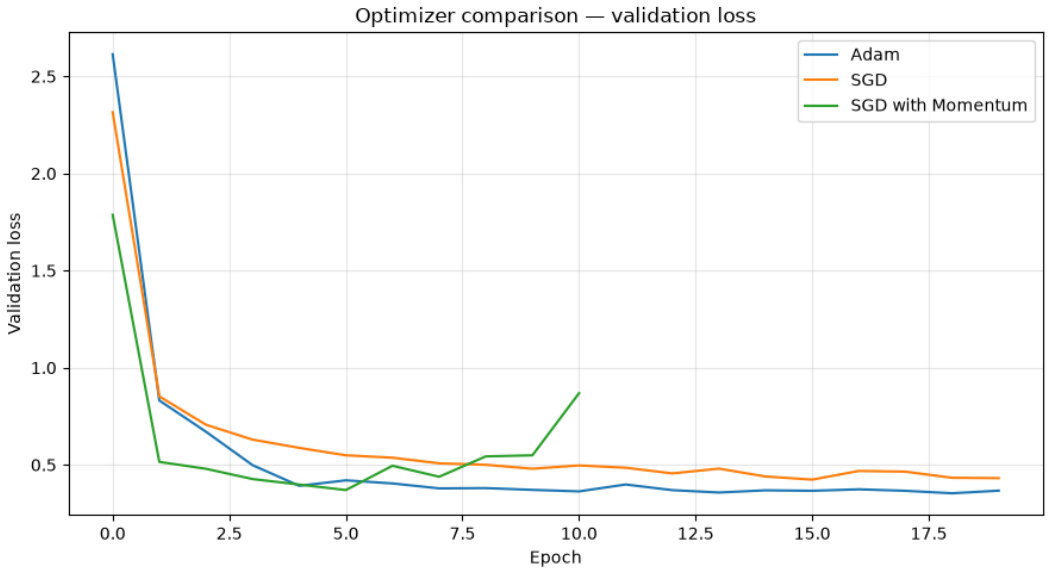


Figure 2: Optimizer comparison plot showing validation loss.

4.2 Training configuration

Both custom CNNs completed the required minimum of 20 epochs. The following settings are taken from the executed notebooks.

Table 6: Training configuration

Setting	Model A	Model B
Loss function	Sparse categorical cross-entropy	Sparse categorical cross-entropy
Optimizer	Adam	Adam
Learning rate	0.001	0.001
Batch size	128	128
Completed epochs	20	25
Checkpointing / early stopping	Best validation checkpoint	Best validation checkpoint; LR reduction

Each pretrained model completed five epochs with the backbone frozen and five fine-tuning epochs. The timing results below are averages over these ten epochs; because notebook execution environments may differ, compare their timings cautiously.

5 Experimental Results

5.1 Model A learning curves



Figure 3: Model A training and validation loss and accuracy across epochs.

5.2 Model B learning curves

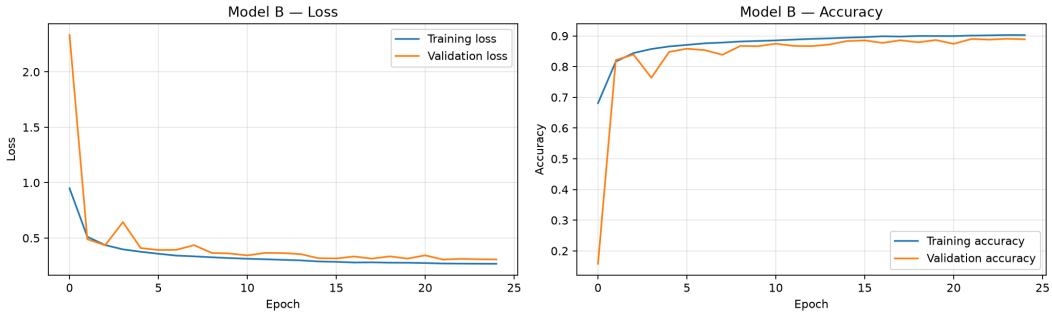


Figure 4: Model B training and validation loss and accuracy across epochs.

5.3 Learning Curve Analysis

Model A shows signs of overfitting, as its training loss decreases while its validation loss increases in later epochs. In contrast, Model B maintains improving performance with a smaller gap between training and validation results. EfficientNetB0 also shows steady improvement, while MobileNetV2's validation performance begins to level off. Overall, Model B offers a good balance between accuracy and model complexity.

5.4 Test-set classification performance

All four models were evaluated on the same held-out test set of 10,500 images. The metrics below are taken from the saved notebook outputs.

Table 7: Test-set classification metrics

Model	Accuracy (%)	Weighted precision	Weighted recall	Macro precision	Macro recall
Model A	89.58	0.8963	0.8958	0.8963	0.8958
Model B	89.10	0.8902	0.8910	0.8902	0.8910
MobileNetV2	88.23	0.8815	0.8823	0.8815	0.8823
EfficientNetB0	89.29	0.8921	0.8929	0.8921	0.8929

Precision measures how many predictions assigned to a class are correct; recall measures how many true examples of a class are retrieved. Weighted averages account for class support, while macro averages give each class equal weight. The test split contains exactly 1,050 examples from each class, so weighted and macro precision/recall are equal (up to rounding) for these results.

5.5 Confusion matrices

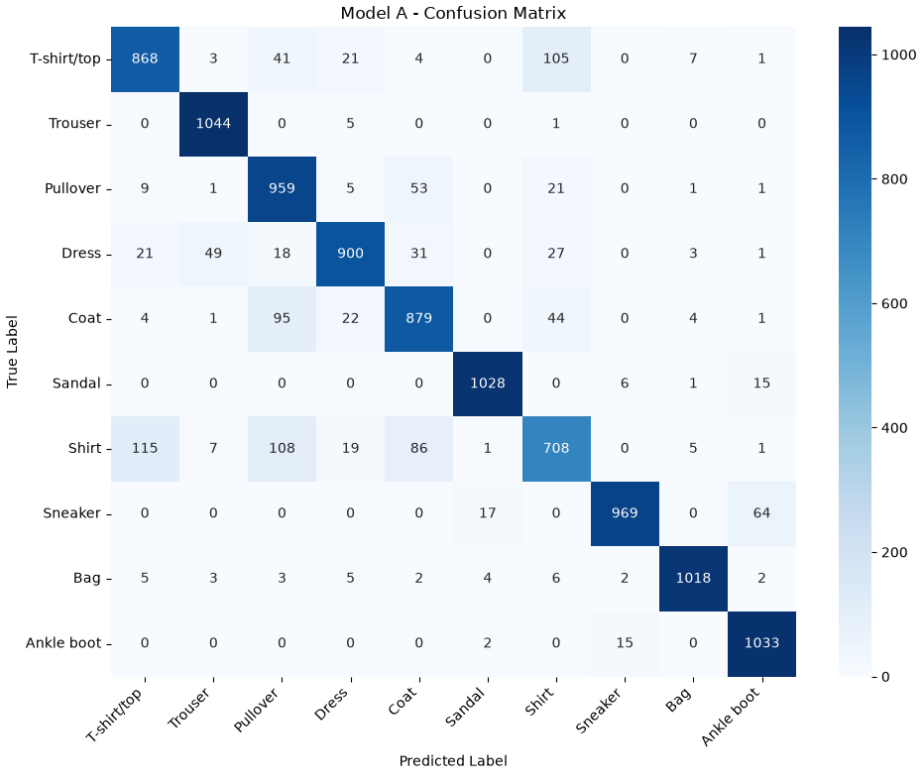


Figure 5: Model A test-set confusion matrix.

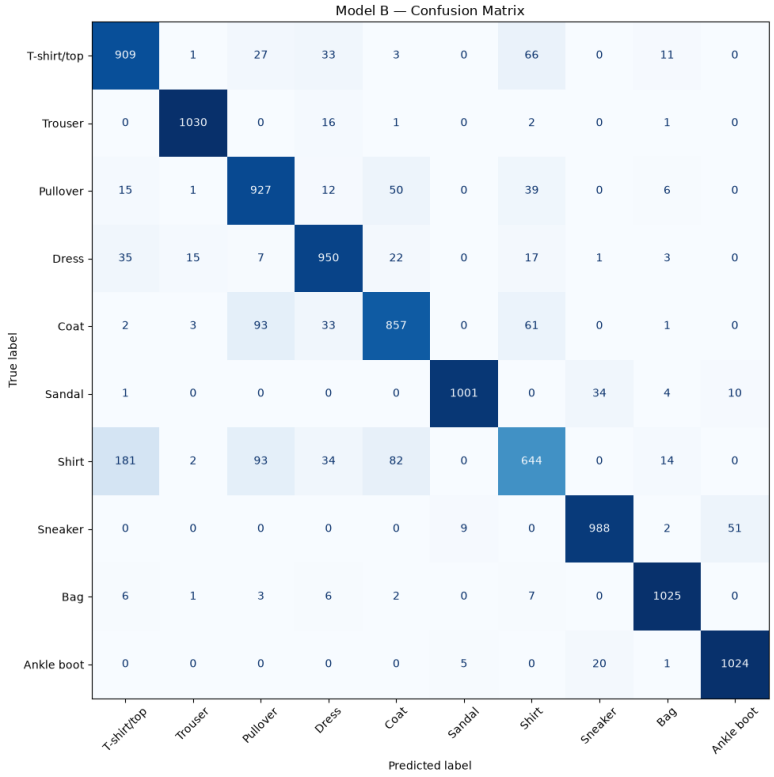


Figure 6: Model B test-set confusion matrix.

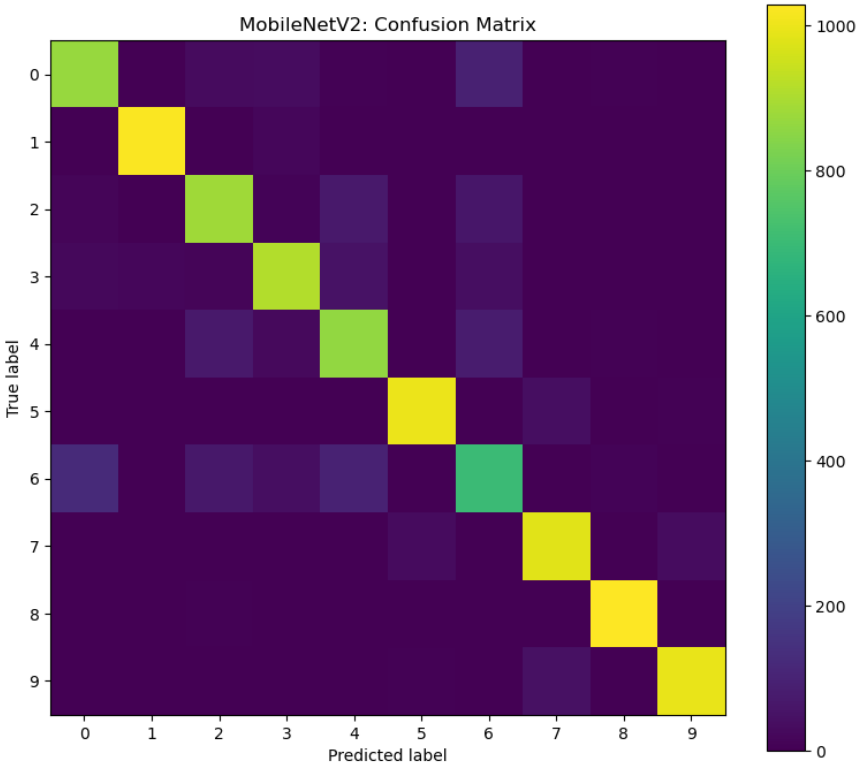


Figure 7: MobileNetV2 test-set confusion matrix.

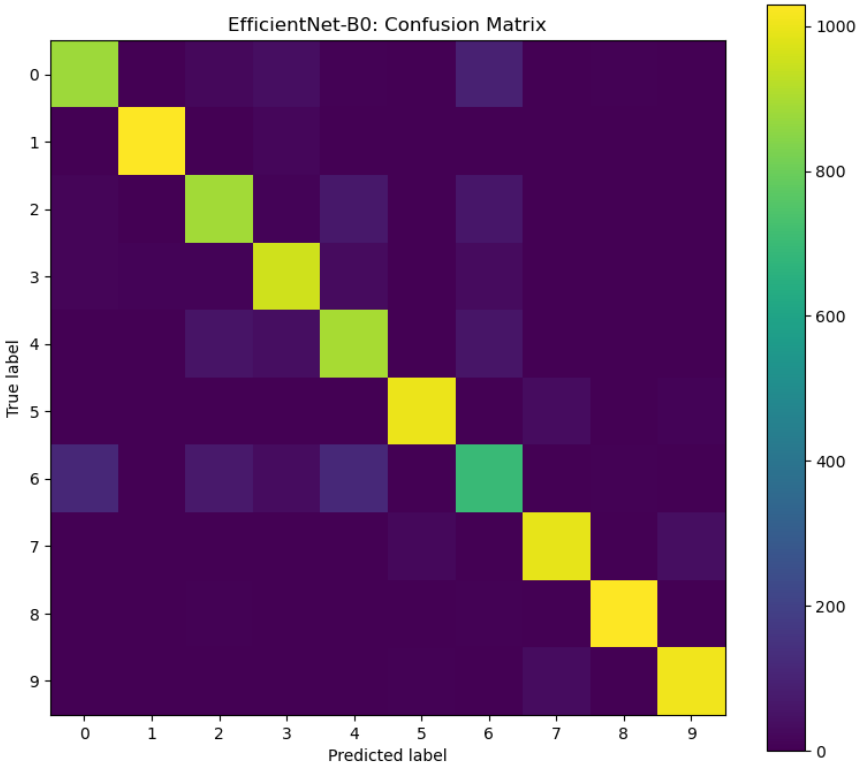


Figure 8: EfficientNetB0 test-set confusion matrix.

5.6 Pretrained model learning curves

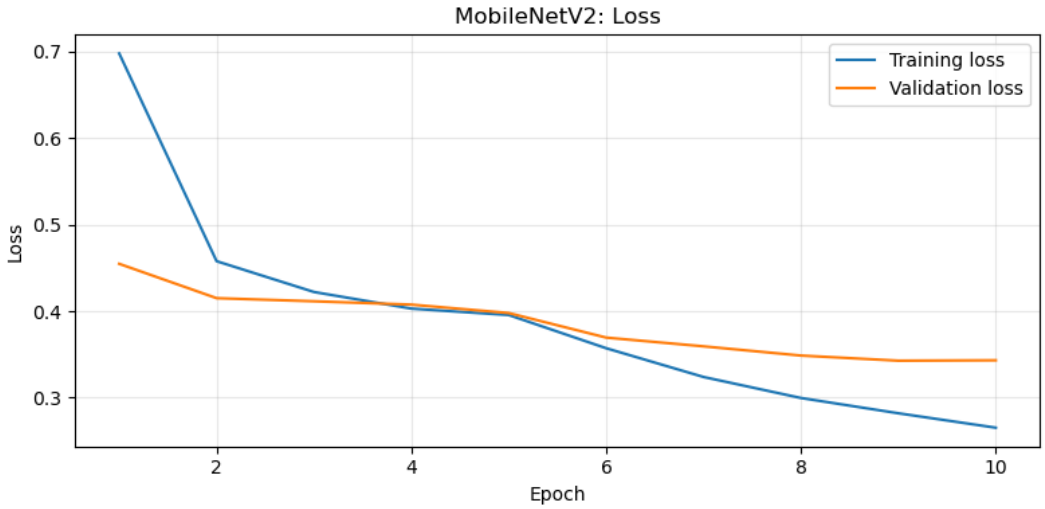


Figure 9: MobileNetV2 training and validation loss curves.

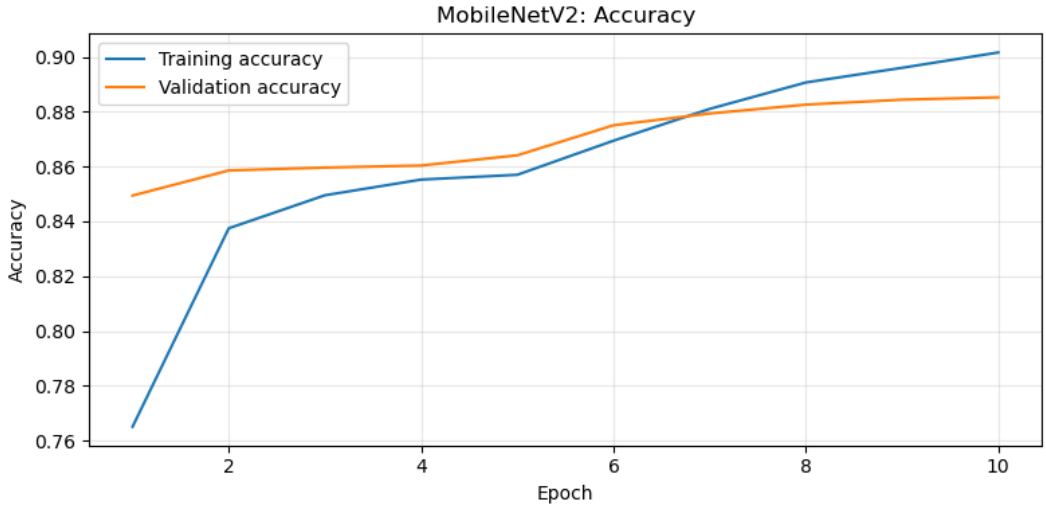


Figure 10: MobileNetV2 training and validation accuracy curves.

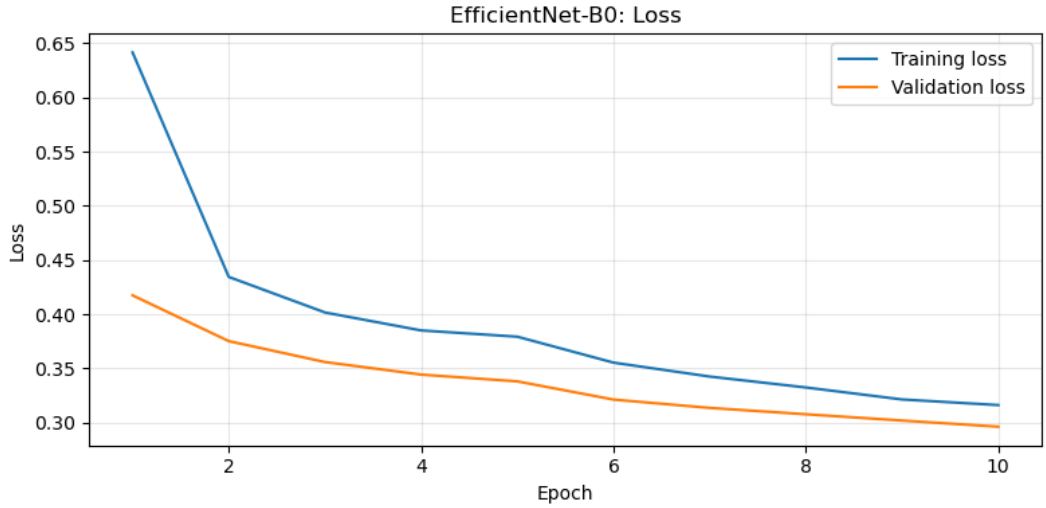


Figure 11: EfficientNetB0 training and validation loss curves.

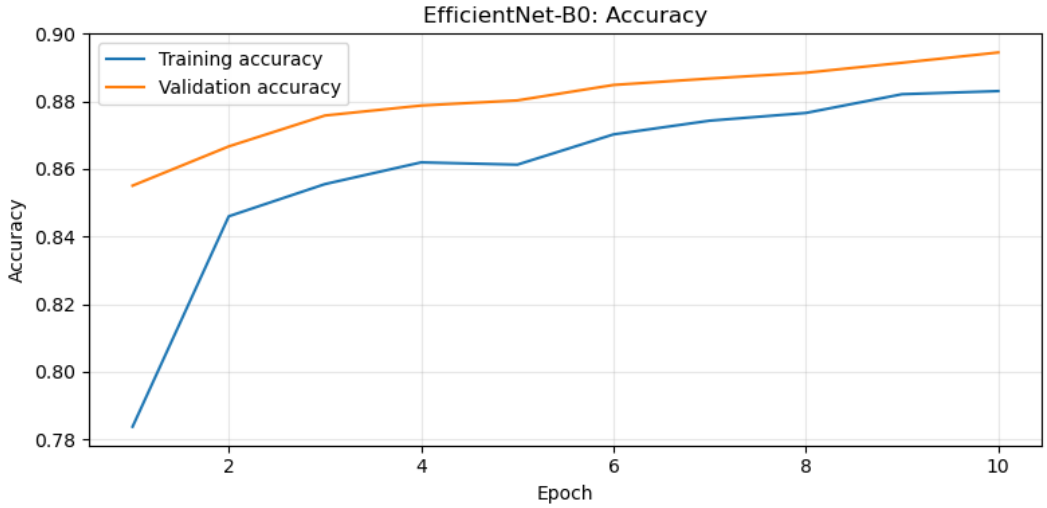


Figure 12: EfficientNetB0 training and validation accuracy curves.

6 Resource Efficiency and Model Comparison

6.1 Custom model comparison

Table 8: Required comparison of the two custom CNNs

Metric	Model A	Model B
Total parameters	60,874	4,738
Estimated/saved model size (KB)	757.90	18.51
Average training time per epoch (s)	17.17	7.52
Test accuracy (%)	89.58	89.10

Model size comparison: Model A's size (757.90 KB) is measured from its saved `.keras` file, whereas Model B's size (18.51 KB) is estimated from its float32 parameter footprint. Therefore, these values are not directly comparable. For a fair comparison, both models should be saved and measured using the same method.

Model B achieved 89.10% test accuracy, close to Model A's 89.58%. However, Model B uses only 4,738 parameters, compared with 60,874 for Model A, reducing the parameter count by approximately 92.22%.

Model B also had a shorter measured training time per epoch (7.52 seconds compared with 17.17 seconds for Model A). These results show that a lightweight CNN can achieve competitive accuracy with far fewer parameters, making it a promising choice for resource-constrained devices.

However, the model-size figures are not directly comparable: Model A's size is measured from its saved file, while Model B's size is an estimated parameter footprint. Therefore, further testing using the same measurement method is needed for a fair comparison.

6.2 Comparison with lightweight pretrained networks

Table 9: Comparison of Model B with pretrained lightweight models

Metric	Model B	MobileNetV2	EfficientNetB0
Total parameters	4,738	2,270,794	4,062,381
Trainable parameters after fine-tuning	4,354	1,207,690	1,355,578
Weight size / saved file (MB)	0.018*	18.447	26.658
Average training time per epoch (s)	7.52	40.39	61.17
Test accuracy (%)	89.10	88.23	89.29
Weighted precision	0.8902	0.8815	0.8921
Weighted recall	0.8910	0.8823	0.8929
Input resolution	$28 \times 28 \times 1$	$64 \times 64 \times 3$	$64 \times 64 \times 3$

6.3 Accuracy, memory and computation trade-offs

1. **Highest test accuracy:** Model A achieved the highest test accuracy of 89.58%, outperforming Model B (89.10%) by 0.48 percentage points.

2. **Model size:** Model B has only 4,738 parameters, compared with 2,270,794 for MobileNetV2 and 4,062,381 for EfficientNetB0. Its estimated float32 weight footprint is only 18.51 KB. The notebook-reported saved sizes for MobileNetV2 and EfficientNetB0 are 18.447 MB and 26.658 MB, respectively.

3. **Effect of pretraining:** ImageNet pretraining did not improve accuracy over Model A. MobileNetV2 achieved 88.23%, while EfficientNetB0 achieved 89.29%. However, EfficientNetB0 performed slightly better than Model B.

4. **Training time:** Model B had the shortest measured custom-model epoch time (7.52 s). MobileNetV2 and EfficientNetB0 averaged 40.39 s and 61.17 s per epoch in their recorded runs. The pretrained-model notebooks ran in a different environment, so these timings are indicative rather than a controlled hardware comparison. Training time alone does not determine inference speed or energy use, which depend on the hardware and implementation.

5. **Model A versus Model B:** Model B achieved nearly the same accuracy as Model A while reducing the parameter count by approximately 92.22%. This makes it a useful option when memory is limited, with only a small reduction in test accuracy.

6. **Overall comparison:** Model B offers a compact and efficient architecture, while Model A provides the highest test accuracy. Pretrained models have substantially more parameters but can benefit from previously learned features. The best choice depends on the application's accuracy and resource requirements.

The experiments show that a smaller model can achieve competitive accuracy without requiring a large pretrained architecture. The custom-model timings were recorded by the notebooks using elapsed wall-clock time around model fitting. The CPU model and RAM were not recorded, and the pretrained notebooks ran in a separate environment; therefore, absolute timing comparisons should be treated cautiously.

7 Limitations

Fashion-MNIST is a low-resolution dataset with ten classes, so results may not generalize to more complex real-world imagery. The pretrained networks receive resized three-channel inputs, whereas custom CNNs can operate directly on grayscale images; this difference should be considered when interpreting computation and input-processing overhead.

Training time depends on CPU/GPU hardware, runtime configuration, batch size, software versions and whether data loading is included. Saved model size depends on serialization format. Parameter count, file size and training time do not alone measure on-device inference latency, peak RAM use, energy consumption or multiply-accumulate operations.

8 Conclusion

This assignment compared two custom CNN architectures with two pretrained image-classification networks on Fashion-MNIST. Model A provided a conventional CNN baseline, while Model B used depthwise separable convolutions to reduce the parameter count.

Model A achieved the highest test accuracy of 89.58%, followed by EfficientNetB0 at 89.29%, Model B at 89.10%, and MobileNetV2 at 88.23%. With only 4,738 parameters, Model B achieved competitive accuracy while using far fewer parameters than the pretrained models. Overall, the results show that a carefully designed lightweight CNN can achieve comparable performance with much lower model complexity, making it a practical choice when memory and computational resources are limited.

9 Reproducibility and GitHub Repository

GitHub repository: https://github.com/Saruka03/EN3150-CNN-Classification
The repository is available at the URL above. For reproducibility, it should include the five Jupyter notebooks, the processed dataset or instructions to recreate it, model-output assets, and a README describing dependencies and execution order.

Hardware used: Notebook logs report no available GPU for the Model B run; CPU model and RAM were not recorded in the exported outputs.

Software versions: TensorFlow 2.21.0 is recorded in the Model B notebook; other package versions were not consistently recorded.

10 References

1. H. Xiao, K. Rasul and R. Vollgraf, "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms," 2017. Available: https://github.com/zalandoresearch/fashion-mnist.

2. A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017. Available: https://arxiv.org/abs/1704.04861.

3. M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018. Available: https://arxiv.org/abs/1801.04381.

4. M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML, 2019. Available: https://arxiv.org/abs/1905.11946.

5. TensorFlow, "Transfer learning and fine-tuning." Available: https://www.tensorflow.org/guide/keras/transfer_learning.

6. TensorFlow, "Keras Applications." Available: https://www.tensorflow.org/api_docs/python/tf/keras/applications.

7. EN3150 Pattern Recognition, "Assignment 03: Resource-Constrained CNN for Edge Image Classification," course assignment brief, University of Moratuwa, 2026.